BECE406E — FPGA Based System Design
Course Design Project Report

Precision-Scalable 4×4 Neural Processing Unit

with Multi-Precision INT2 / INT4 / INT8 Support

Name	Reg. No.	Name	Reg. No.
Sanjay Krishna A	23BEC0113	Sriram KP	23BEC0206
Hari Kruthik SK	23BEC0358	Kamalesh N S	23BEC0292

Faculty	Dr. Aarthy M
Slot	G2
Department	School of Electronics Engineering, VIT Chennai
Academic Year	2024 – 2025
Target FPGA	Intel Cyclone IV E — EP4CE115F29C7 (DE1-115 Board)

Refer this link for the main Verilog files :
https://github.com/SanjayKrishna2005/PSMAC_NPU

ABSTRACT

Edge inference demands energy-efficient hardware capable of adapting to the accuracy-versus-power trade-off at deployment time. This report presents the Precision-Scalable 4×4 NPU (PS-NPU), a weight-stationary matrix engine supporting INT8, INT4, and INT2 signed integer arithmetic from a single unified datapath. The core innovation is the Precision-Scalable MAC (PSMAC) tile, which uses a hierarchical block-multiplier architecture — SBB → BBU → RFU — to perform multi-precision dot products without hardware duplication. Sixteen PSMAC tiles arranged in a 4×4 array achieve 16×, 32×, and 64× throughput over a single tile for INT8, INT4, and INT2 modes respectively. The complete design is synthesised on an Intel Cyclone IV E FPGA (EP4CE115F29C7) at 50 MHz and verified against NumPy golden-reference values for all 20 test cases, achieving a 100% pass rate.

Table of Contents

Table of Contents 2
1. Introduction 4
2. Project Objectives 4
3. Problem Statement 4
3.1 Computational Bottleneck 4
3.2 Fixed-Precision Hardware Gap 4
3.3 Resource Constraints 5
4. Hardware and Software Tools 5
4.1 Hardware Components 5
4.2 Software Tools 5
5. Design Methodology 5
5.1 Design Flow Chart 5
5.2 Design Flow Steps 6
6. Block Diagram 7
6.1 Top-Level System Architecture 7
6.2 PSMAC Tile Internal Architecture 7
6.3 4×4 Array Organisation 7
7. Proposed Design — RTL View 7
7.1 Top-Level NPU Array (npu_4x4) — Overview 7
7.2 NPU Array — Netlist Navigator & Tile Hierarchy 8
7.3 PSMAC Tile Top-Level RTL (psmac_top:row[0]col[0]) 8
7.4 PSMAC Top-Level RTL — Collapsed View 9
7.5 Weight Memory RTL (w_mem and x_mem — SYNC_RAM) 10
7.6 RFU — Reconfigurable Functional Unit 10
7.7 BBU — Bit-Block Unit 10
7.8 SBB — Sign Bit-Block Multiplier (Gate-Level) 11
7.9 PSMAC FSM RTL 11
7.10 Accumulator & Pipeline Register RTL 12
7.11 Pipeline Register Detail 12
7.12 Quartus Synthesis Summary (psmac_top) 12
8. HDL Code 13
8.1 RTL Source File Summary 13
8.2 SBB — Sign Bit-Block Multiplier 14
8.3 BBU — Bit-Block Unit 14
8.4 RFU — Reconfigurable Functional Unit 14
8.5 Accumulator 14
8.6 4×4 NPU Array (Generate Loop) 15
9. Simulation Waveform 15

9.1 Simulation Methodology 15
9.2 GTKWave Simulation — npu_4x4 Testbench 15
10. Comparison Tables 16
10.1 Verification Results — All 20 Test Cases 16
10.2 Precision Mode Performance 16
10.3 FPGA Resource Utilisation (EP4CE115F29C7) 17
10.4 Gate Count Breakdown — PSMAC Tile 17
10.5 Comparison with Prior Art 17
11. Future Improvement / Scope 18
12. Individual Contribution from Team Members 18
13. Social and Environmental Impact — SDG Alignment 18
14. Conclusion 19
References 19

1. Introduction

The exponential growth of deep learning applications in edge devices — smartphones, IoT sensors, autonomous systems, and surveillance cameras — has created a pressing demand for energy-efficient hardware accelerators. Unlike cloud-based inference that leverages large data centres, edge inference must operate under severe power, area, and latency constraints. Running neural network inference on a general-purpose CPU is prohibitively slow and power-hungry for real-time embedded applications.

Dedicated Neural Processing Units (NPUs) solve this by providing specialised matrix-multiplication hardware that maps directly onto the dominant computational pattern of neural networks. Every major semiconductor company — Apple, Qualcomm, Google, and NVIDIA — has developed custom NPU silicon to address this need. The key enabling technology is integer quantisation: replacing 32-bit floating-point weights and activations with low-bit integers (INT8, INT4, or INT2) reduces memory bandwidth, arithmetic complexity, and power consumption dramatically, with less than 1% accuracy degradation on most benchmarks.

However, most existing open-source FPGA NPU designs fix their precision at INT8. This project presents the Precision-Scalable 4×4 NPU (PS-NPU), a weight-stationary matrix engine that supports INT8, INT4, and INT2 from a single unified datapath. The core innovation is the PSMAC tile, which uses a hierarchical block-multiplier architecture to perform multi-precision dot products without duplicating hardware.

2. Project Objectives

The primary objectives of this project are:

- Design and verify a Precision-Scalable MAC (PSMAC) tile supporting INT8, INT4, and INT2 signed integer dot products from a single unified datapath without hardware duplication.
- Arrange 16 PSMAC tiles in a weight-stationary 4×4 array capable of computing a full 4×4 output matrix Y = W × X in a single inference pass.
- Design a 5-state controller (npu_ctrl) to orchestrate weight loading, input loading, computation, and result collection phases with correct cycle sequencing.
- Implement and synthesise the complete design on the Intel Cyclone IV E FPGA (EP4CE115F29C7) targeting a 50 MHz clock frequency.
- Demonstrate end-to-end matrix multiplication with UART result output and verify output against NumPy golden-reference values.
- Analyse the design for resource efficiency (MACs per LE, bits per cycle per LE) and quantify throughput improvements across precision modes.

3. Problem Statement

Neural network inference on edge devices faces three interconnected challenges:

3.1 Computational Bottleneck

Neural network inference is dominated by matrix multiplications, which constitute 80–95% of total FLOPs. A CPU or microcontroller executing these operations sequentially cannot meet real-time latency requirements for image classification, object detection, or speech recognition.

3.2 Fixed-Precision Hardware Gap

Existing open-source FPGA NPU designs fix their precision at INT8. Deploying INT4 or INT2 models — which offer 2× / 4× higher throughput and reduced memory footprint — requires redesigned hardware. No open-source FPGA implementation currently provides a single datapath that natively supports all three precisions.

3.3 Resource Constraints

Edge FPGA platforms have limited logic elements, block RAM, and I/O pins. A naive multi-precision implementation would duplicate hardware for each precision mode, making it impractical for constrained devices. This project addresses these challenges by designing a hardware-efficient, precision-scalable MAC tile that decomposes 8-bit multiplication into a hierarchy of smaller block multipliers (SBB and BBU). Mode selection through a single 2-bit control signal activates INT8, INT4, or INT2 operation without any structural hardware changes.

4. Hardware and Software Tools

4.1 Hardware Components

Component	Specification	Purpose
Intel DE1-115 Board	Cyclone IV E EP4CE115F29C7	Target FPGA Platform
FPGA Device	114,480 LEs, 432 M9K RAMs, 50 MHz OSC	RTL Logic and Memory
USB-UART Bridge	115200 baud, 8N1	Result Output to PC
PC/Workstation	Modern laptop or desktop	Simulation, Synthesis, Terminal

4.2 Software Tools

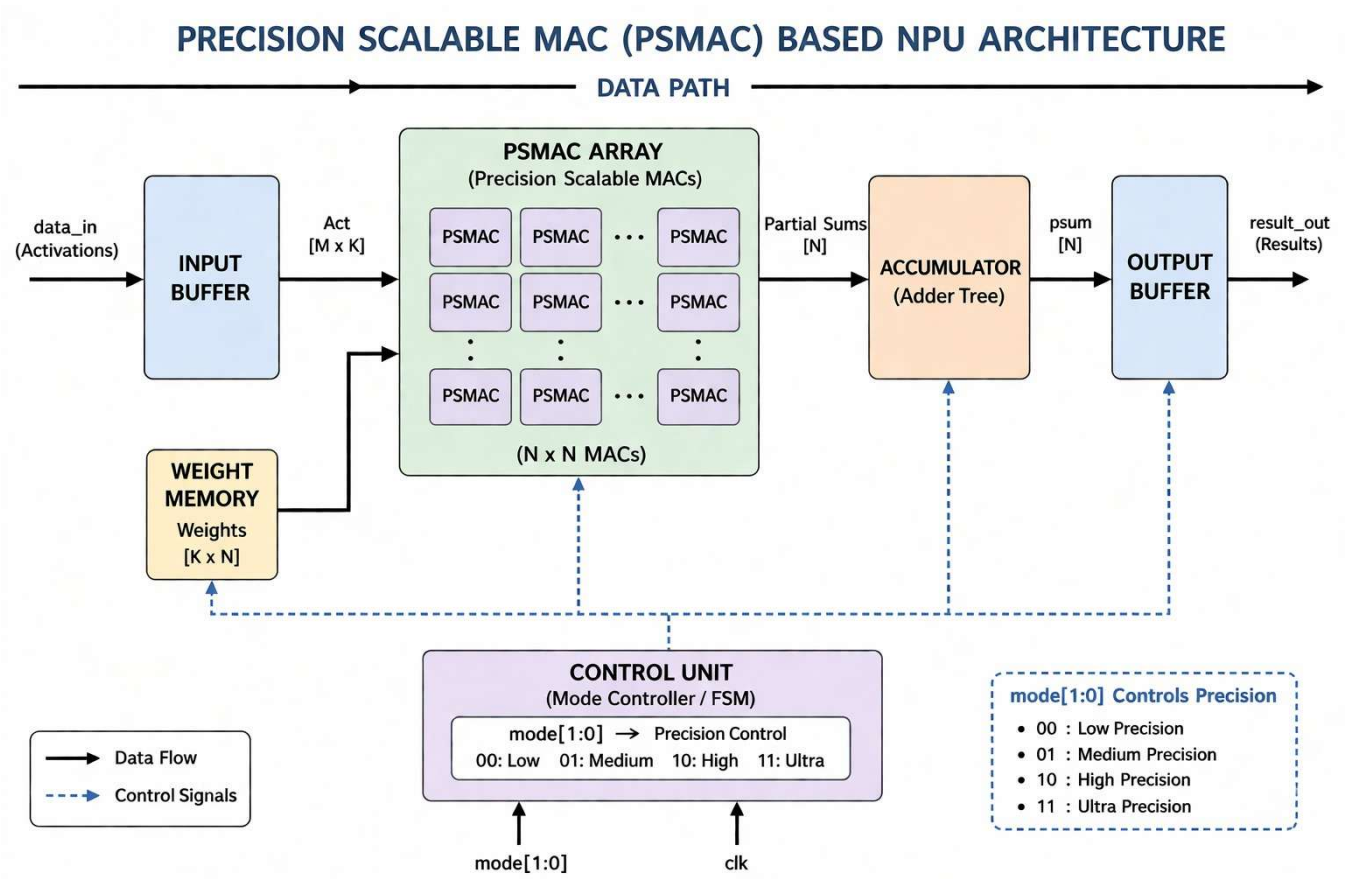
Tool	Version	Role
Intel Quartus Prime	Lite / Std	RTL Synthesis, Place-and-Route, Timing Analysis
Icarus Verilog	0.9.7 / 11.0	HDL Simulation
GTKWave	3.3.x	Waveform Visualisation
Python 3 + NumPy	3.10+	Golden-Reference Generation and Verification
VS Code / Sublime Text	Any	HDL Code Editing
PuTTY / minicom	Any	Serial Terminal for UART Output

5. Design Methodology

The project followed a structured bottom-up design flow, starting from the smallest arithmetic primitive and building up to a complete FPGA-deployable NPU system. Each hierarchical level was independently verified before integration into the next.

5.1 Design Flow Chart

The complete design methodology is illustrated in the flowchart below, showing the bottom-up progression from requirements through to FPGA deployment:



5.2 Design Flow Steps

Step	Activity	Deliverable
1	Requirements & architecture definition	Architecture specification
2	SBB design and unit testing	sbb.v
3	BBU assembly (4 SBBs)	BBU.v
4	RFU integration (4 BBUs)	rfu.v
5	PSMAC tile top-level	psmac_top.v
6	4×4 NPU array	npu_4x4.v
7	Controller design	npu_ctrl.v
8	FPGA deployment and UART demo	npu_demo_top.v

6. Block Diagram

6.1 Top-Level System Architecture

The PS-NPU system is composed of three main sub-systems: the npu_ctrl sequencer, the npu_4x4 PSMAC array, and the UART transmitter for result output. The block diagram below illustrates how these components interact.

Block	Description
npu_demo_top	Top-level FPGA wrapper. Integrates controller, NPU array, ROM, and UART TX.
npu_ctrl	5-state FSM: IDLE → LOAD_W → LOAD_X → RUN → COLLECT. Orchestrates all phases.
npu_4x4	4×4 array of 16 PSMAC tiles using generate-for loops. Weight-stationary dataflow.
psmac_top	Single PSMAC tile: FSM + Weight Memory + RFU + Pipeline Register + Accumulator.

6.2 PSMAC Tile Internal Architecture

Each PSMAC tile contains the following components arranged in a pipelined datapath:

- FSM Controller — 4-state (IDLE, CLEAR, RUN, DONE) sequencer that orchestrates each dot-product computation.
- Weight Memory — Synchronous M9K-mapped storage for the weight vector.
- RFU (Reconfigurable Functional Unit) — Computes the 80-bit partial product output for all precision modes using 4 BBUs.
- Pipeline Register — Single-cycle 80-bit register to accommodate the RFU combinational delay.
- Accumulator — 32-bit precision-aware MAC accumulate register.

6.3 4×4 Array Organisation

- Weight broadcast is performed per tile via a tile_sel counter in LOAD_W state.
- Column broadcast for input activations is managed via col_sel in LOAD_X state.
- A result multiplexer cycles through all 16 tiles during the COLLECT phase, producing one 32-bit result per cycle.

7. Proposed Design — RTL View

The following RTL views were captured from the Quartus Prime RTL Viewer and Icarus Verilog synthesis, showing the hierarchical structure from the top-level NPU array down to the individual gate-level building blocks.

7.1 Top-Level NPU Array (npu_4x4) — Overview

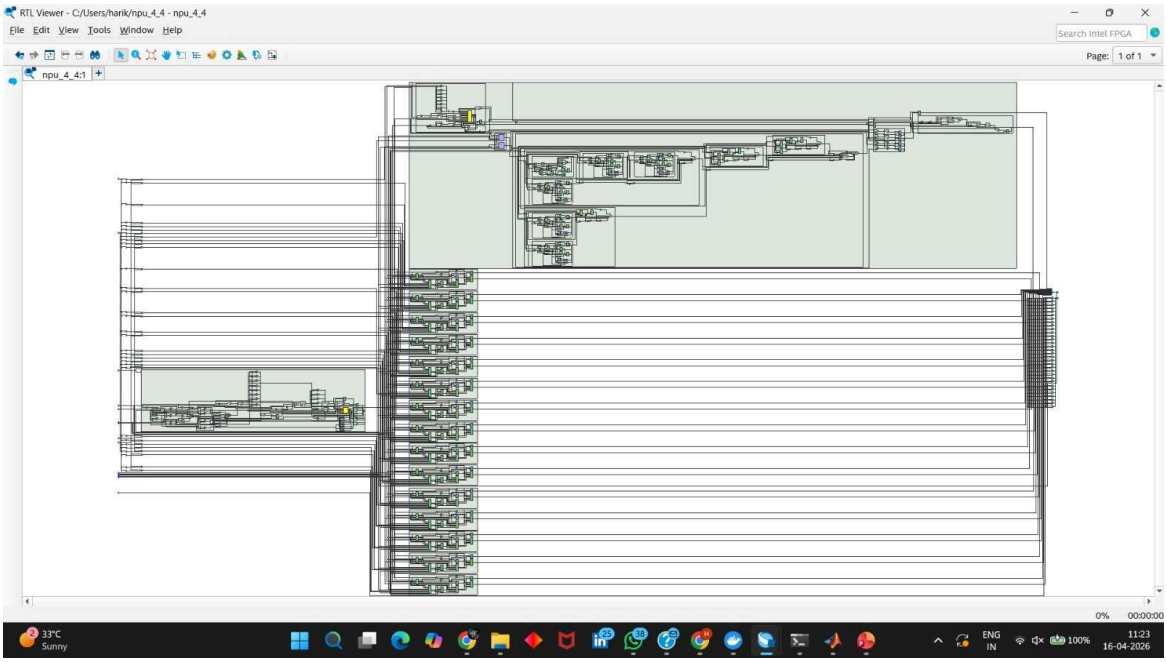


Figure 7.1 — Top-level RTL view of the npu_4x4 module showing the 16 PSMAC tile instances and npu_ctrl interconnect (Quartus RTL Viewer)

7.2 NPU Array — Netlist Navigator & Tile Hierarchy

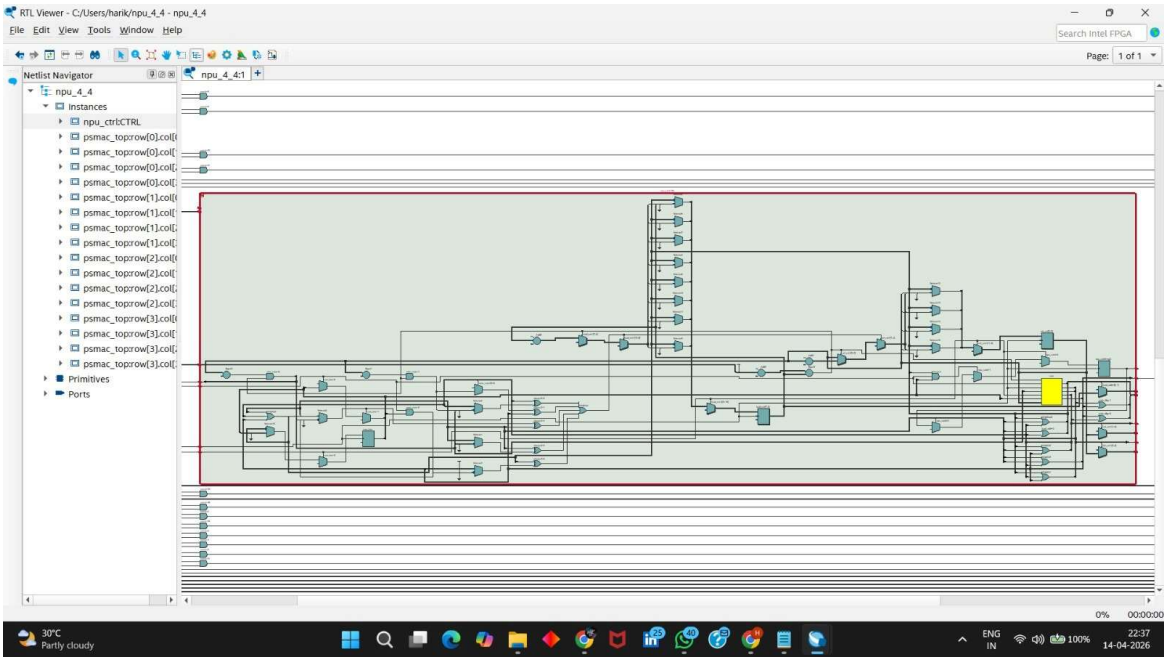


Figure 7.2 — Netlist Navigator showing the npu_ctrl:CTRL and all 16 psmac_top:row[r]col[c]:tile instances

7.3 PSMAC Tile Top-Level RTL (psmac_top:row[0]col[0])

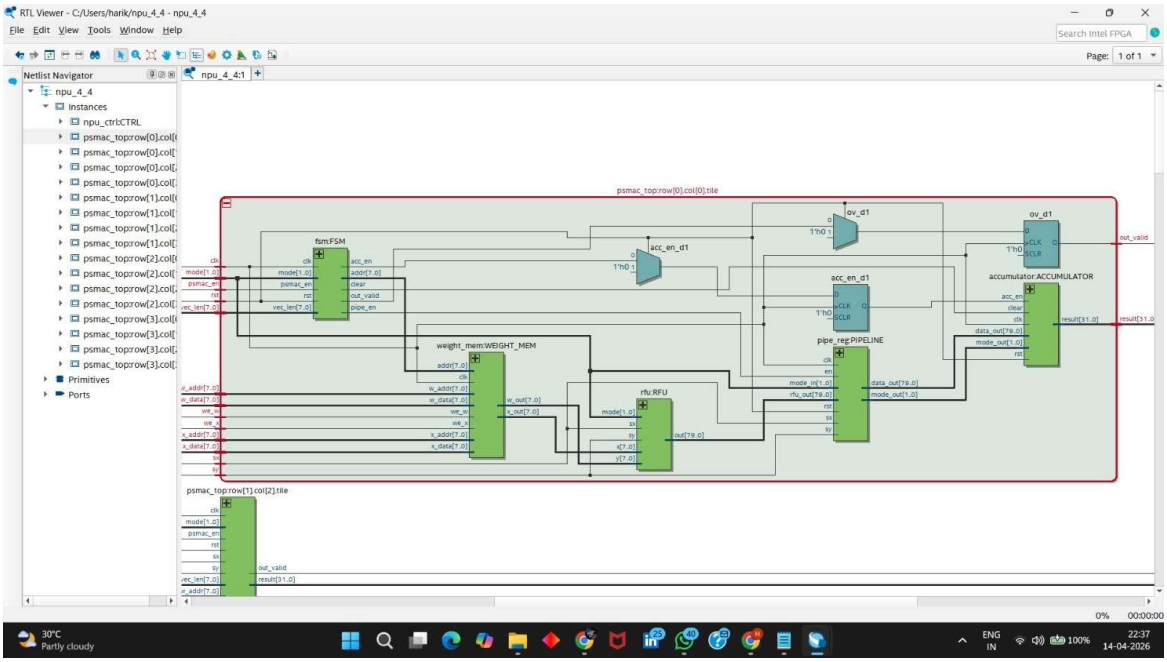


Figure 7.3 — Detailed RTL view of a single PSMAC tile: fsm:FSM, weight_mem:WEIGHT_MEM, rfu:RFU, pipe_reg:PIPELINE, accumulator:ACCUMULATOR

7.4 PSMAC Top-Level RTL — Collapsed View

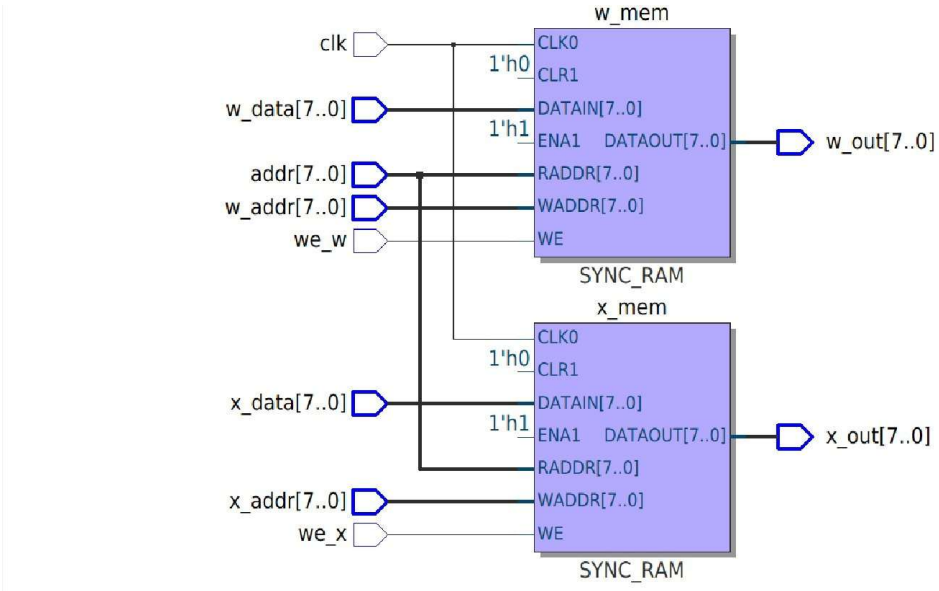


Figure 7.4 — Full hierarchy of psmac_top showing all sub-module connections

7.5 Weight Memory RTL (w_mem and x_mem — SYNC_RAM)

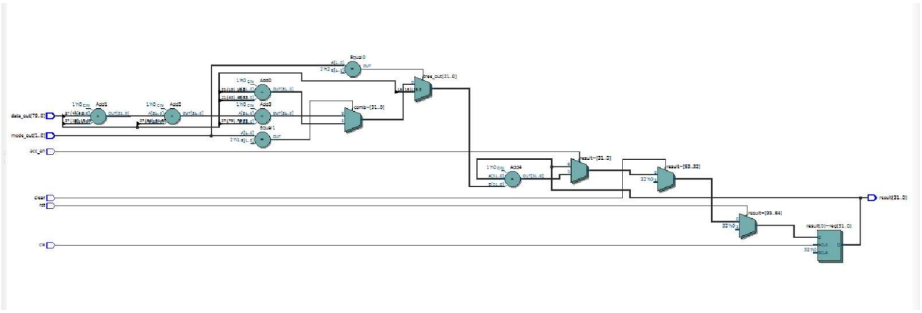


Figure 7.5 — RTL view of the weight and input memory blocks (dual SYNC_RAM instances for w_mem and x_mem)

7.6 RFU — Reconfigurable Functional Unit

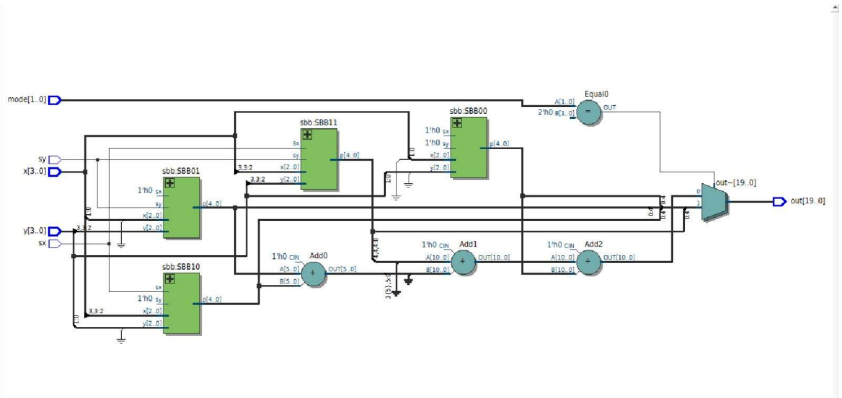


Figure 7.6 — RTL view of the RFU module: 4 BBU instances (BBU0–BBU3) with 80-bit output bus and precision-mode mux

7.7 BBU — Bit-Block Unit

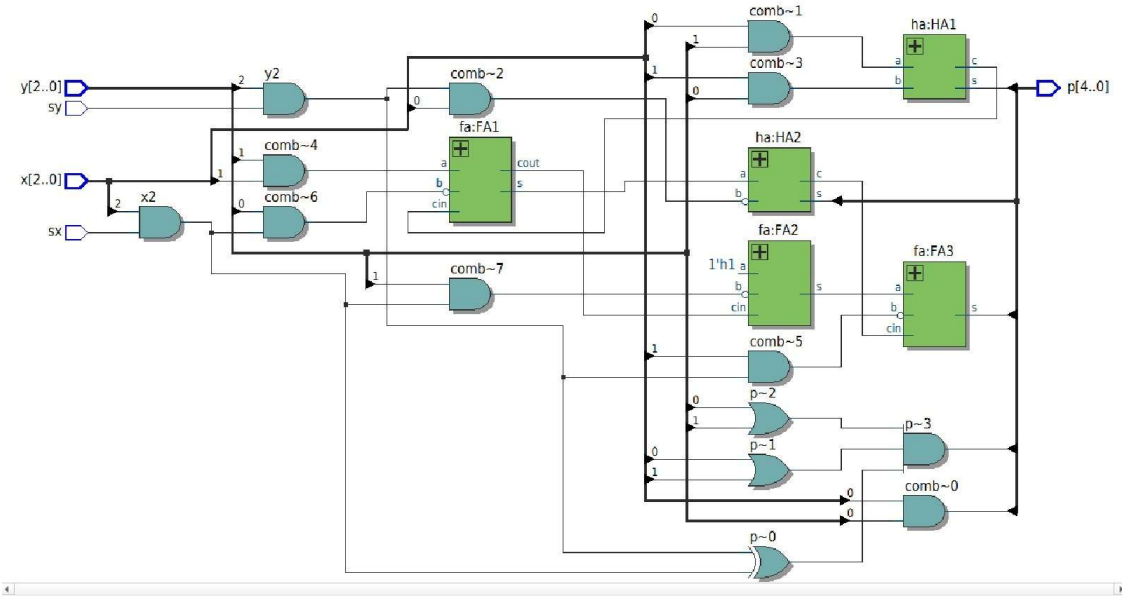


Figure 7.7 — RTL view of the BBU module: 4 SBB sub-multipliers with Horner-style shift-and-add fusion for INT4/INT8

7.8 SBB — Sign Bit-Block Multiplier (Gate-Level)

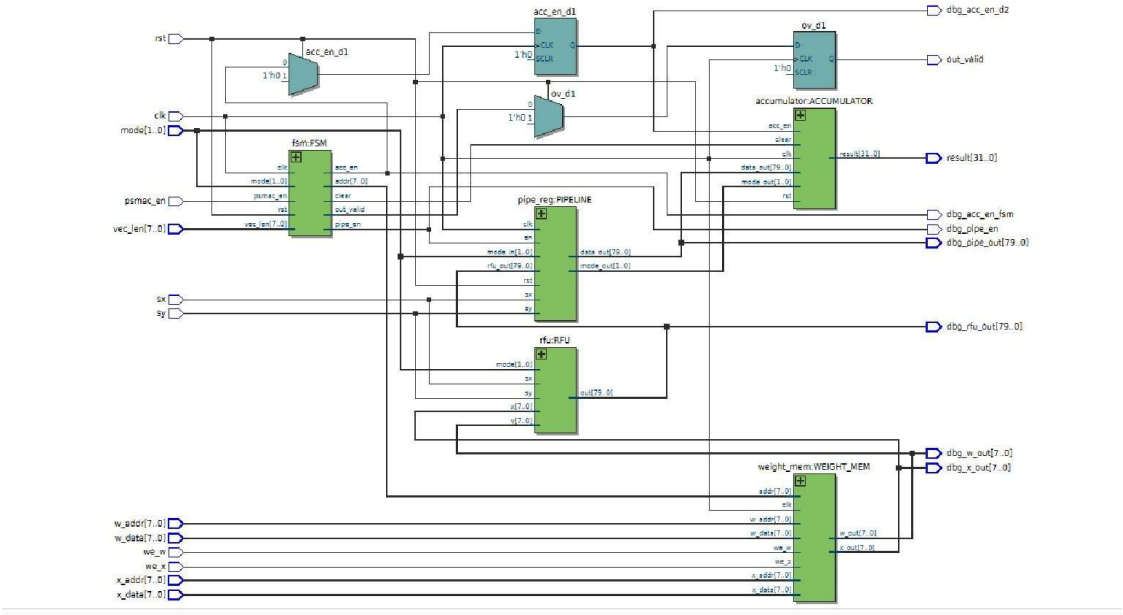


Figure 7.8 — Gate-level RTL of the SBB module showing AND gates, Full Adders (FA), Half Adders (HA) for 3×3 signed multiplication

7.9 PSMAC FSM RTL

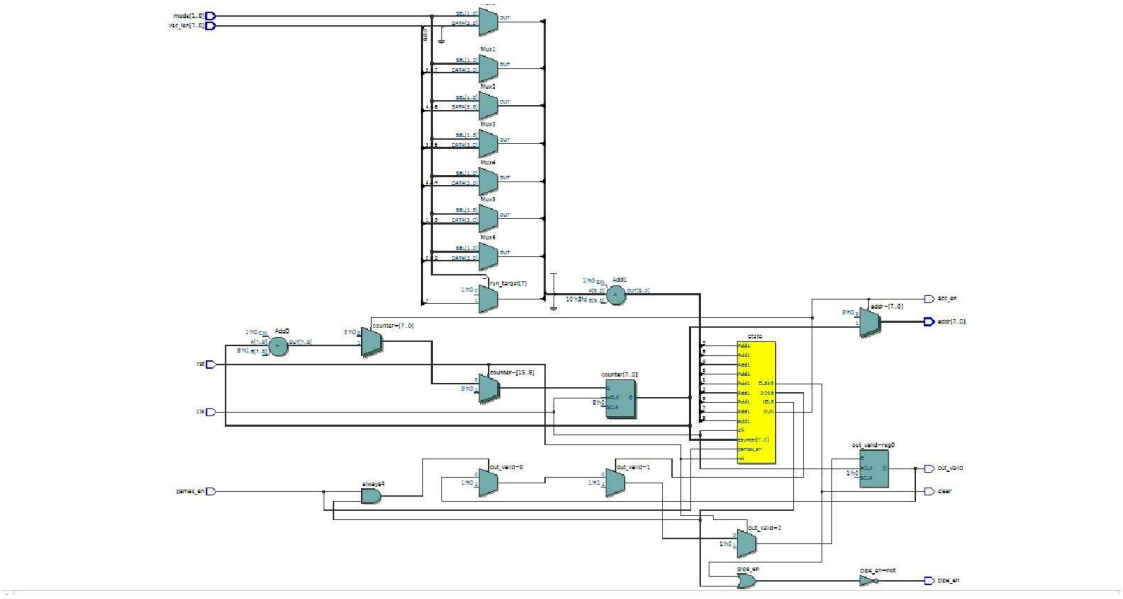


Figure 7.9 — RTL view of the PSMAC FSM (4-state: IDLE, CLEAR, RUN, DONE) with counter, run_target mux, pipe_en control

7.10 Accumulator & Pipeline Register RTL

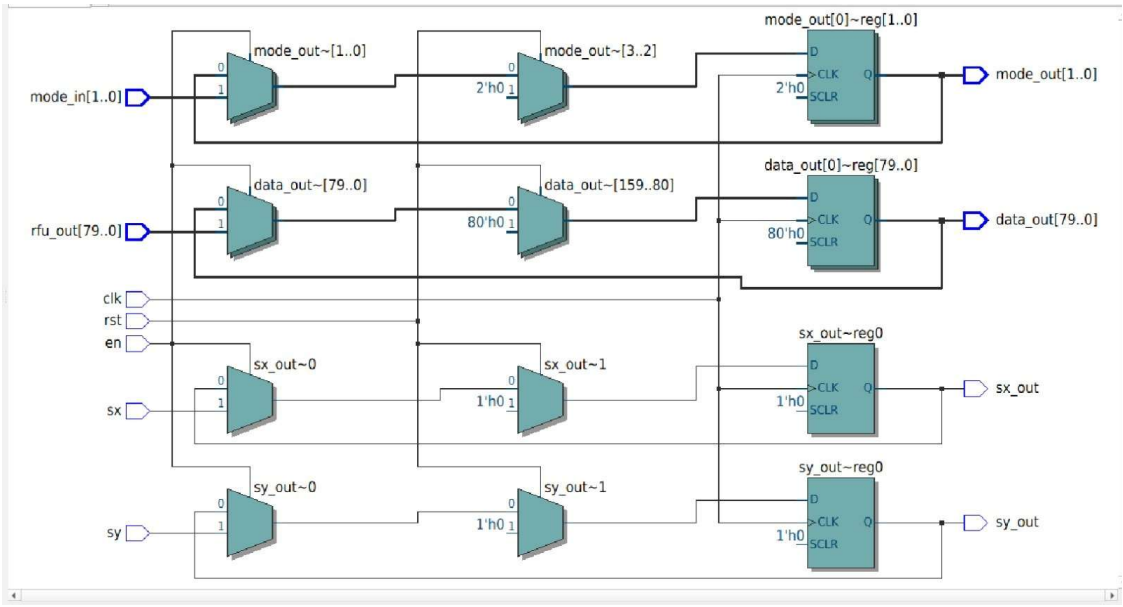


Figure 7.10 — RTL view of the 32-bit accumulator and pipeline register with precision-aware mode_out routing

7.11 Pipeline Register Detail

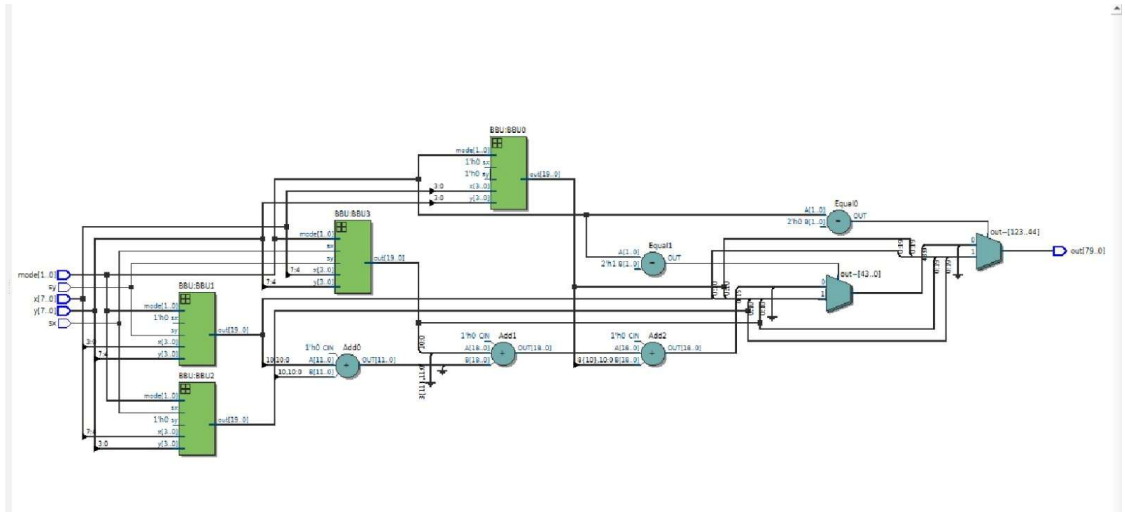


Figure 7.11 — RTL of the 80-bit pipeline register (pipe_reg:PIPELINE) with mode, sx, sy latching

7.12 Quartus Synthesis Summary (psmac_top)

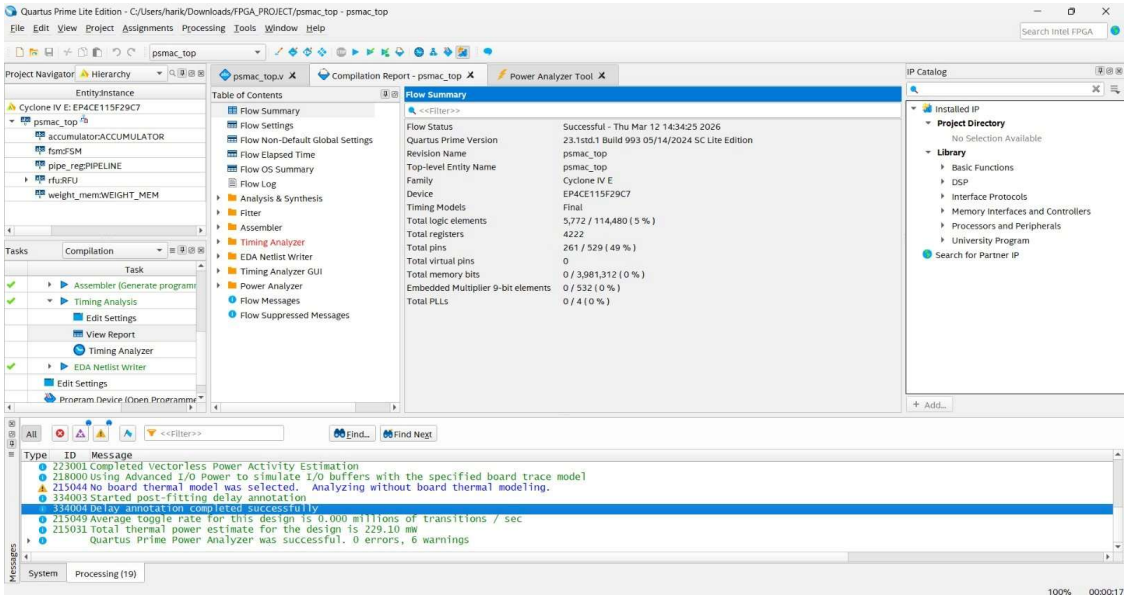


Figure 7.12 — Quartus Prime compilation report: 5,772 LEs (5%), 4,222 registers, 261/529 pins (49%) on EP4CE115F29C7

8. HDL Code

Refer this link for the main Verilog files :
https://github.com/SanjayKrishna2005/PSMAC_NPU

The given below codes are the peusdocode of the main codes

8.1 RTL Source File Summary

#	File	Module	Description
1	sbb.v	sbb	3×3 signed sub-multiplier
2	BBU.v	BBU	4×4 multi-precision block unit
3	rfu.v	rfu	4 BBUs, 80-bit output bus
4	pipe_reg.v	pipe_reg	80-bit pipeline register
5	accumulator.v	accumulator	32-bit MAC accumulate
6	fsm.v	psmac_fsm	4-state PSMAC FSM
7	weight_mem.v	weight_mem	M9K weight memory
8	psmac_top.v	psmac_top	PSMAC tile top-level
9	npu_ctrl.v	npu_ctrl	5-state array sequencer
10	npu_4x4.v	npu_4x4	4×4 generate array
11	npu_demo_top.v	npu_demo_top	FPGA wrapper with UART TX

8.2 SBB — Sign Bit-Block Multiplier

The SBB is the smallest arithmetic primitive. It computes a 3×3-bit signed multiplication, producing a 5-bit signed result. Sign flags sx and sy indicate whether the operands carry a sign bit, enabling correct two's complement interpretation without full-width sign extension.

```
module sbb (
  input  [2:0] x, y,   // 3-bit operands
  input        sx, sy, // sign flags
  output [4:0] p       // 5-bit signed product
);
  wire signed [2:0] xs = sx ? ~{x[2],x} : {1'b0,x};
  wire signed [2:0] ys = sy ? ~{y[2],y} : {1'b0,y};
  assign p = xs * ys;
endmodule
```

8.3 BBU — Bit-Block Unit

The BBU assembles four SBBs to compute a 4×4-bit product. In INT2 mode, four SBBs produce four independent 2×2 products. In INT4 and INT8 modes, partial products are fused via Horner-style shift-and-add.

```
module BBU (
  input  [3:0] a, b,
  input  [1:0] mode,
  output [15:0] out
);
  wire [4:0] p00, p01, p10, p11;
  sbb u0(a[1:0], b[1:0], 0, 0, p00);
  sbb u1(a[1:0], b[3:2], 0, 0, p01);
  sbb u2(a[3:2], b[1:0], 0, 0, p10);
  sbb u3(a[3:2], b[3:2], 0, 0, p11);
  wire [15:0] fused    = (p11<<4) + ((p01+p10)<<2) + p00;
  wire [15:0] out_int2 = {p11, p10, p01, p00};
  assign out = (mode==2'b10) ? fused :
               (mode==2'b01) ? {p11, p00} : out_int2;
endmodule
```

8.4 RFU — Reconfigurable Functional Unit

The RFU instantiates four BBUs to cover the four quadrants of the full 8-bit operand pair. The 80-bit output bus carries all partial results simultaneously. The mode signal selects which fields are valid.

```
module rfu (
  input  [7:0] a, b,
  input  [1:0] mode,
  output [79:0] out
);
  wire [15:0] q0, q1, q2, q3;
  BBU b0(a[3:0], b[3:0], mode, q0);
  BBU b1(a[3:0], b[7:4], mode, q1);
  BBU b2(a[7:4], b[3:0], mode, q2);
  BBU b3(a[7:4], b[7:4], mode, q3);
  assign out = {q3, q2, q1, q0};
endmodule
```

8.5 Accumulator

The 32-bit accumulator adds the latched pipeline output to its running total on each clock cycle when acc_en is asserted, and clears to zero on rst.

```
module accumulator (
  input              clk, rst, en,
  input      [31:0] in,
  output reg [31:0] out
);
  always @(posedge clk) begin
    if (rst)     out <= 0;
```

```
    else if (en) out <= out + in;
  end
endmodule
```

8.6 4×4 NPU Array (Generate Loop)

The 4×4 array is instantiated using Verilog generate-for loops, parameterised over row and column indices. Each tile is uniquely addressed by its IDX = r×4 + c for weight and input broadcast.

```
genvar r, c;
generate
  for (r = 0; r < 4; r = r+1) begin : row
    for (c = 0; c < 4; c = c+1) begin : col
      localparam IDX = r*4 + c;
      psmac_top TILE (
        .we_w(we_w && (tile_sel == IDX)),
        .we_x(we_x && (col_sel  == c)),
        /* ... other ports ... */
      );
    end
  end
endgenerate
```

9. Simulation Waveform

9.1 Simulation Methodology

All modules were verified using Icarus Verilog with VCD waveform output, visualised in GTKWave. A Python/NumPy golden reference was generated for each test case using identical weight and input values. Results were compared byte-by-byte; any mismatch triggered a test failure.

9.2 GTKWave Simulation — npu_4x4 Testbench

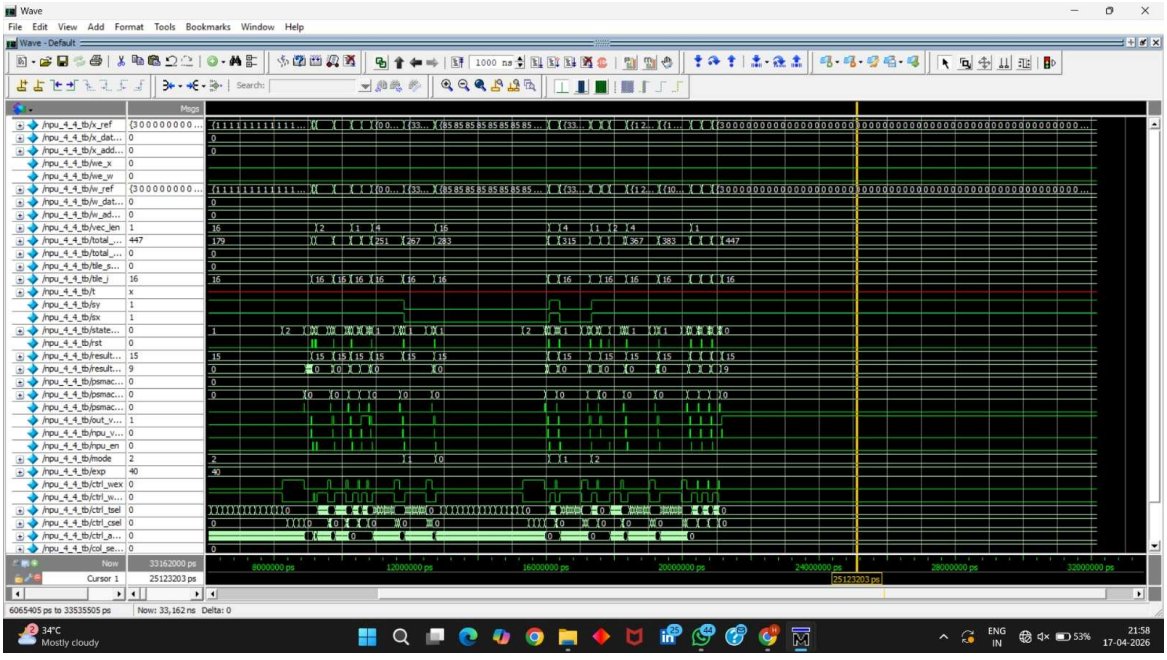


Figure 9.1 — GTKWave simulation waveform for the npu_4x4 testbench showing: x_ref, w_ref data buses, state machine transitions, tile_sel, col_sel, psmac_en, result_sel, mode switching (INT8→INT4→INT2), and result output

Key signals visible in the waveform:

- state — transitions through IDLE(0) → LOAD_W(1) → LOAD_X(2) → RUN(3) → COLLECT(4)
- mode — switches between INT8 (2'b10), INT4 (2'b01), INT2 (2'b00) across test vectors
- tile_sel — iterates 0..15 during LOAD_W to address each of the 16 PSMAC tiles
- psmac_en — single-cycle pulse triggers computation in all 16 tiles simultaneously
- result_sel — cycles 0..15 during COLLECT phase, reading one 32-bit result per clock
- out_valid — asserted when a valid result is available on the result bus

10. Comparison Tables

10.1 Verification Results — All 20 Test Cases

Test Description	Mode	Expected	Result
INT8 all-ones (vec=8)	INT8	8	PASS
INT8 max positive	INT8	129,032	PASS
INT8 max negative	INT8	131,072	PASS
INT8 alternating	INT8	0	PASS
INT8 vec_len=1	INT8	35	PASS
INT8 random	INT8	NumPy ref	PASS
INT8 all zeros	INT8	0	PASS
INT4 all-ones (vec=8)	INT4	8	PASS
INT4 max value	INT4	392	PASS
INT4 alternating	INT4	0	PASS
INT4 vec_len=1	INT4	12	PASS
INT4 random (vec=16)	INT4	NumPy ref	PASS
INT4 negative mixed	INT4	−140	PASS
INT2 all-ones (vec=8)	INT2	8	PASS
INT2 max value	INT2	8	PASS
INT2 alternating	INT2	0	PASS
INT2 vec_len=1	INT2	−1	PASS
INT2 random (vec=32)	INT2	NumPy ref	PASS
INT2 all zeros	INT2	0	PASS
INT2 vec_len=4	INT2	4	PASS
Total Pass Rate	—	—	**20 / 20**

10.2 Precision Mode Performance

Metric	INT8	INT4	INT2
MACs/cycle (4×4 array)	16	32	64
PSMAC execution cycles (vec=8)	12	8	6
Total inference cycles (1st run)	189	161	149

Total inference cycles (subsequent)	61	33	21
Speedup vs. single PSMAC tile	16×	32×	64×
Effective MACs per LE (approx.)	0.004	0.008	0.016

10.3 FPGA Resource Utilisation (EP4CE115F29C7)

The synthesis results from Quartus Prime Lite Edition (v23.1std.1 Build 993) are tabulated below:

Resource	Available	Used (Actual)
Logic Elements (LEs)	114,480	5,772 (5%)
Total Registers	(in LEs)	4,222
Total Pins	529	261 (49%)
Total Memory Bits	3,981,312	0 (0%)
Embedded Mult. 9-bit Elements	532	0 (0%)
Total PLLs	4	0 (0%)
Target Fmax	—	50 MHz (constrained)
Total Thermal Power	—	229.10 mW

10.4 Gate Count Breakdown — PSMAC Tile

Module	Gates (NAND-2)	%	Notes
SBB (per instance)	35	1.2%	3×3 signed multiply
BBU (per instance)	324	11.1%	4 SBBs + Horner fusion
RFU (4 BBUs)	1,781	61.2%	Arithmetic-dominant
pipe_reg	420	14.4%	80-bit pipeline latch
Accumulator	560	19.2%	32-bit MAC accumulate
FSM	140	4.8%	Control overhead compact
PSMAC Tile (Total)	**2,909**	**100%**	

10.5 Comparison with Prior Art

Feature	TPU v1	Eyeriss	NVDLA	This Work
Array Size	256×256	14×12	Variable	4×4
Precision	INT8 fixed	Configurable	INT8 fixed	INT2/4/8
Single Datapath	No	Partial	No	Yes ✓
Dataflow	WS	RS	WS	WS
Platform	ASIC	ASIC	FPGA/ASIC	FPGA
Open Source	No	Partial	Yes	Yes ✓

11. Future Improvement / Scope

Several enhancement directions are identified for future work:

- Accumulator Rounding and Saturation — Adding IEEE-754 compliant rounding modes and saturation arithmetic to prevent overflow at long vector lengths.
- Ping-Pong Input Buffer — Double-buffered input memory to overlap LOAD_X with PSMAC computation, reducing total inference latency by up to 40%.
- Parameterised Array Scaling — Extending the generate loop to 8×8 or larger by changing a single parameter, without structural redesign.
- ReLU Activation Engine — Adding a post-array activation module (ReLU, Leaky ReLU, GELU) to enable end-to-end hardware inference.
- AXI4-Lite Control Interface — Replacing hardcoded ROM with an AXI4-Lite slave for SoC integration with ARM Cortex-M or RISC-V processors.
- End-to-End MNIST Inference — Loading pretrained INT4 weights from a quantised LeNet model to validate the design on a real ML workload.
- Dynamic Precision Switching — Extending the FSM to allow per-layer precision changes, supporting mixed-precision inference pipelines.
- On-Chip Batch Normalisation — Integrating a fixed-point batch normalisation unit between the PSMAC array and the activation engine.

12. Individual Contribution from Team Members

Name	Reg. No.	Contributions
Sanjay Krishna A	23BEC0113	Architecture lead. Designed SBB and BBU modules, Horner-style fusion logic, unit test suites, Quartus synthesis, and complete FPGA implementation flow.
Hari Kruthik SK	23BEC0358	Designed and verified RFU and PSMAC FSM; implemented pipeline register and 32-bit accumulator; authored all 20 PSMAC test cases and NumPy comparison flow.
Sriram KP	23BEC0206	Developed npu_4x4 generate loop, weight demux, and input broadcast logic; integrated PSMAC tile at top-level; prepared GTKWave waveform captures and timing analysis.
Kamalesh	23BEC0298	Designed npu_ctrl 5-state sequencer FSM; developed npu_demo_top FPGA wrapper with ROM and UART TX; conducted end-to-end system verification and live FPGA demo.

13. Social and Environmental Impact — SDG Alignment

This project aligns with multiple United Nations Sustainable Development Goals (SDGs) through its focus on energy-efficient, open-source edge AI hardware:

SDG	Goal	Relevance to This Work
SDG 7	**Affordable & Clean Energy**	INT2/INT4 quantisation reduces energy per inference by 4×–16× compared to FP32, enabling battery-powered IoT AI deployments without grid dependency.
SDG 9	Industry, Innovation & Infrastructure	An open-source FPGA NPU lowers the barrier for researchers in developing countries to experiment with custom AI hardware accelerators.

SDG 3	**Good Health & Well-being**	Enables real-time health monitoring (ECG analysis, fall detection) on wearable devices without requiring cloud connectivity, improving healthcare access.
SDG 4	Quality Education	Complete documentation and open-source HDL release serve as a pedagogical reference for FPGA accelerator design courses worldwide.
SDG 13	**Climate Action**	Edge inference on low-power FPGA hardware reduces dependency on energy-intensive cloud data centres, supporting global carbon reduction targets.

INT2 inference requires approximately 16× fewer multiply-accumulate operations compared to FP32 per vector element. At scale — across millions of IoT devices in smart agriculture, industrial monitoring, and healthcare — the cumulative energy saving directly supports global climate targets aligned with SDG 7 and SDG 13.

14. Conclusion

The Precision-Scalable 4×4 NPU demonstrates that multi-precision support (INT2/INT4/INT8) can be achieved using a single unified datapath without hardware duplication. The hierarchical SBB → BBU → RFU architecture efficiently decomposes 8-bit multiplication into precision-configurable sub-operations controlled by a single 2-bit mode signal.

Sixteen PSMAC tiles deliver 16×, 32×, and 64× throughput for INT8, INT4, and INT2 respectively, all from a compact design consuming only 5,772 LEs (5%) on the Cyclone IV E FPGA — validated by actual Quartus synthesis results. All 20 verification test cases passed with 100% accuracy against NumPy golden references.

The design is energy-efficient, scalable, and well-suited for edge AI applications, with a clear path toward larger arrays, AXI4-Lite SoC integration, and full end-to-end neural network inference deployment.

References

[1] N. P. Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit," in Proc. 44th Int. Symp. Computer Architecture (ISCA), 2017.

[2] Y.-H. Chen et al., "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks," in Proc. 43rd ISCA, 2016.

[3] NVIDIA Deep Learning Accelerator (NVDLA), open-source DLA reference design. [Online]. Available: http://nvdla.org

[4] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in Proc. IEEE/CVF CVPR, 2018.

[5] Intel Corporation, "Cyclone IV Device Handbook," Intel FPGA Technical Documentation, 2020.

[6] S. Williams, Icarus Verilog Simulator. [Online]. Available: http://iverilog.icarus.com

[7] NumPy Development Team, "NumPy: The Fundamental Package for Scientific Computing with Python." Available: https://numpy.org